ALGORITHM 2: Optimal-DPOR algorithm.

Initial call: $Explore(\langle\rangle, \emptyset, \langle\{\langle\rangle\}, \emptyset\rangle)$

```
1  Explore(E, Sleep, WuT)
2      if enabled(s_[E]) = ∅ then                          // Race detection only at maximal execution sequences
3          foreach e, e' ∈ dom(E) such that (e ≲_E e') do  // For each racing pair e, e'
4              let E' = pre(E, e);                         // Goto state before e
5              let v = notdep(e, E).ê';                    // Find events independent with e
6              if sleep(E') ∩ WI_[E'](v) = ∅ then          // Has an equivalent already been explored?
7                  wut(E') := insert_[E'](v, wut(E'));     // If not, insert into the wakeup tree
8      else                                                // If not at a maximal execution sequence, explore...
9          if WuT ≠ ⟨{⟨⟩}, ∅⟩ then
10             wut(E) := WuT;                              // ... either using an existing wakeup tree
11         else
12             choose p ∈ enabled(s_[E]);                  // ... or by selecting an arbitrary p...
13             wut(E) := ⟨{⟨⟩, p}, {(p, ⟨⟩)}⟩;             // ... and making a wakeup tree from it
14         sleep(E) := Sleep;
15         while ∃p ∈ wut(E) do                            // While the wakeup tree is not empty...
16             let p = min_≺{p ∈ wut(E)};                  // ... pick next branch, ...
17             let Sleep' = {q ∈ sleep(E) | E ⊢ p◇q};      // ... compute next sleep set...
18             let WuT' = subtree(wut(E), p);              // ... and wakeup tree (a subtree of the current),...
19             Explore(E.p, Sleep', WuT');                 // ... and do a recursive call to Explore
20             remove all sequences of form p.w from wut(E); // When done, cleanup...
21             add p to sleep(E);                          // ... and mark p as explored
```

races that can be reversed in the just explored sequence E. Such a race consists of two events e and e' in E, such that $e \precsim_E e'$. Let $E' = pre(E, e)$ and let $v = notdep(e, E).\widehat{e'}$, i.e., the sub-sequence of E consisting of the events that occur after e but do not "happen after" e, followed by $\widehat{e'}$ (this notation is introduced at lines 4 and 5). The reversible race $e \precsim_E e'$ indicates that there is another execution sequence, which performs v after E', and in which the race is reversed, i.e., the event e' happens before the event e. Since $E'.v$ is incompatible with the currently explored computation, the algorithm must now make sure that it will be explored if it was not explored previously. If some $p \in sleep(E')$ is in $WI_{[E']}(v)$, then some execution equivalent to one starting with $E'.v$ will have been explored previously. If not, we perform the operation $insert_{[E']}(v, wut(E'))$ to make sure that some execution equivalent to one starting with $E'.v$ will be explored in the future.

In the exploration phase, which is entered if exploration has not reached the end of a maximal execution sequence, first the wakeup tree $wut(E)$ is initialized to WuT. If WuT is empty, then (as we will state in Lemma 7.1) the sleep set is empty, and an arbitrary enabled process is entered into $wut(E)$. The sleep set $sleep(E)$ is initialized to the sleep set that is passed as argument in this call to *Explore*. Thereafter, each sequence in $wut(E)$ is subject to exploration. We find the first (i.e., minimal) single-process branch p in $wut(E)$ and call *Explore* recursively for the sequence $E.p$. In this call, the associated sleep set $Sleep'$ is obtained from $sleep(E)$ in the same way as in Algorithm 1. The associated wakeup tree WuT' is obtained as the corresponding subtree of $wut(E)$. Thereafter, *Explore* is called recursively for the sequence $E.p$ with the modified sleep set $Sleep'$ and wakeup